

Dynamic Frame Balancing: Weighted Shares Reach the Ceiling, Stealing Buys Insurance

Matias Saavedra

Sunday 12th July, 2026

Binary: branch feat/frame-distribution, commits ffe61aa
(DIST_WEIGHTS/DIST_SEED/DIST_FRAMES + dist_dynamic.sh) and 8c21bf4
(weight-proportional chunk cap), atop the report-25 lineage (aae49fb/52c40c4, tag
binary-frame-dist; main 8acde54, v7). Same node pair as 25-frame-distribution.tex; ivy
runs the GPU=0 build. Raw data: experiments/25-frame-distribution/
(results_dynamic.csv, results_dynamic_uncapped.csv).

Resumen

Report 25 ended with an analytic verdict: on the laptop+ivy pair ($4.56\times$ per-frame throughput gap), no static equal-share assignment can beat the laptop alone, and even a perfect weighted split is bounded at $1.22\times$ (15.4 s) with 3.4–4.2 s of orchestration overhead eating half of that — so *parity* is the realistic target. This report builds the two schedulers that verdict calls for and measures them. A seeded weighted-random static assignment (DIST_WEIGHTS, splitmix64 over the frame index — communication-free, identical on every machine) and a coordinator-driven work-stealing mode (guided chunks through the new DIST_FRAMES list interface; an empty node steals half the richest undispatched tail). Both are **100/100 byte-identical** at production scale, including a live corrective steal. With calibrated 5:1 weights both reach **baseline parity** (22.93 ± 0.11 and 22.01 ± 0.56 s vs. a 21.06 ± 2.73 s baseline; equal-share static: 46.90 s). With deliberately wrong 1:1 weights, stealing recovers **72%** of the miscalibration penalty autonomously ($46.90 \rightarrow 28.18 \pm 0.37$ s) — but only after fixing the defect the first run exposed: an in-flight chunk is unstealable, so chunks must be capped at $KCAP \cdot w_r$ frames to bound every node’s exposure at the same wall-time budget.

1. Motivation: what report 25 left on the table

The equal-share benchmark of **25-frame-distribution.tex** was a clean negative: cyclic assignment ran +150% against the laptop alone because both nodes got ~ 50 frames and ivy needs 0.855 s per frame where the laptop hybrid needs 0.188 s. Its analysis derived exactly what a better scheduler is worth on this pair: the combined-throughput ideal is 15.4 s ($1.22\times$), the fast node should carry $\sim 82\%$ of the frames, and 3.4–4.2 s of ship/collect overhead consumes most of the ideal gain. Two mechanisms follow from that arithmetic, and this report implements and measures both:

1. a **static weighted** assignment — give each node a share proportional to its speed, randomly interleaved so every share samples the zoom’s $32\times$ cost trend uniformly (the weighted analogue of cyclic); and

2. **dynamic work stealing** on top — when a node is about to finish, it pulls frames from the loaded node, covering the case where the weights are wrong (they are configuration-dependent: *S5.5* shows the same pair at $2.4\times$, not $4.56\times$, when the laptop is CPU-pinned).

2. The change: two ownership modes and a stealing coordinator

Weighted-random static (DIST_WEIGHTS, commit ffe61aa).

A third ownership mode joins report 25's block-cyclic predicate in `main.cpp` (precedence: `DIST_FRAMES > DIST_WEIGHTS > block-cyclic`):

Código 1: `main.cpp` — the weighted-random owner draw (abridged).

```

1 // SplitMix64: the weighted-random frame owner must be computed
2 // identically by every rank on every machine and compiler, so the
3 // draw is pure uint64 arithmetic -- no floating point, no rand().
4 unsigned long long h =
5     splitmix64 (distSeed + (unsigned long long) i)
6     % (unsigned long long) distWeightSum;
7 owned = h >= (unsigned long long) distWeightLo
8     && h < (unsigned long long) distWeightHi;
```

Frame i belongs to the rank whose cumulative-weight interval $[lo_r, hi_r)$ contains the hash — with `DIST_WEIGHTS=5,1`, rank 0 owns each frame with probability $5/6$. Because the draw is a pure integer function of `(seed, i)`, every rank on every machine computes the *same partition* with no communication, preserving report 25's collision-free-union property; and because it is (hash-)random in i , each share is an unbiased sample of the cost trend. Everything downstream is untouched.

The explicit-list mode (DIST_FRAMES).

The second addition is the dispenser interface report 25's seam predicted: `DIST_FRAMES="3,17,42"` renders exactly those global indices (range-checked, no `DIST_NODES/RANK` needed). It is the primitive the dynamic coordinator drives.

The stealing coordinator (scripts/dist_dynamic.sh).

One process on the coordinator; no node-to-node sockets; the intra-node region queue untouched. The frame list is shuffled with a seeded deterministic shuffle and dealt proportionally to `WEIGHTS` into per-rank *bags*. Each node's driver loops: take a guided chunk from its own bag — half the remaining frames, at least `KMIN` (4), at most `KCAP · wr` (*S5.2*) — run one `DIST_FRAMES` invocation on the node, collect its PNGs, repeat. When a node's bag is empty — it is about to finish — the driver **steals** half the richest node's undispached tail (same caps) and keeps rendering. A single **flock** serializes all bag mutations, and an in-flight chunk is already out of every bag, so each frame is rendered exactly once; guided halving keeps a node's own share at $\sim \log_2(\text{share}/\text{KMIN})$ process startups.

3. Methodology

Same pair, spec, and discipline as `25-frame-distribution.tex`: node A = laptop (i7-9750H + GTX 1660 Ti, hybrid 12 1 0.1 32768 1 1), node B = ivy (i7-4702MQ, **make GPU=0**, CPU-only 8 0 0.1 32768 1 1); production spec (100 frames, 1920×1080), **save** = 1, both machines on AC (gated by the driver), 3 reps per configuration.

Identity.

Executor-pinned (both nodes CPU-only), mirroring report 25 *S3*: weighted static 5:1 and dynamic-with-stealing 5:1, each **cmp**'d frame-by-frame against the single-process CPU reference.

Benchmark (`bench_dynamic.sh`).

Four configurations: the laptop-alone baseline (re-run in-batch — absolute walls drift between batches, *S5.6*); **weighted_static** 5:1 (one invocation per node via `dist_frames.sh WEIGHTS=5,1`); **dyn_51** (stealing coordinator, calibrated weights); and **dyn_11** (stealing coordinator, deliberately *wrong* equal weights — the robustness probe: how much of equal-share static's +150% does stealing recover with no correct information?). The benchmark ran twice: once before the chunk cap (`results_dynamic_uncapped.csv`) and once with it (`results_dynamic.csv`, the shipped configuration); the first run is reported because it measured the defect the cap fixes. 5:1 approximates the measured $4.56\times$ throughput ratio (the optimal 82/18 split vs. 5:1's 83/17 — indistinguishable at $F = 100$).

4. Results

Identity: weighted static 5:1 (85/15 split) **100/100 byte-identical**; dynamic 5:1 **100/100 byte-identical**, with a live corrective steal (ivy stole 5 frames back from the CPU-pinned laptop — *S5.5*).

Tabla 1: Both benchmark runs: means $\pm \sigma$ over 3 reps. “laptop/ivy” are each node’s own compute wall (static: the rank’s [total_elapsed_s]; dynamic: the sum over its chunks); “shares” are frames actually rendered, with stolen frames in parentheses. Δ compares against the *same batch*’s baseline. The capped run is the shipped configuration; the uncapped run is kept because its **dyn_11** row measures the unstealable-chunk defect.

Run	Config	e2e (s)	laptop (s)	ivy (s)	shares L/I	Δ
uncapped	baseline	19.20 ± 0.55	19.20	—	100/—	—
	weighted static	23.30 ± 0.27	17.45	19.87	85/15	+21%
	dyn 5:1	22.90 ± 1.15	18.47	16.73	83/17 (0 stolen)	+19%
	dyn 1:1	31.27 ± 1.08	16.48	28.01	75/25 (25 stolen)	+63%
capped	baseline	21.06 ± 2.73	21.06	—	100/—	—
	weighted static	22.93 ± 0.11	17.93	19.85	85/15	+8.9%
	dyn 5:1	22.01 ± 0.56	18.96	16.31	83/17 (0 stolen)	+4.5%
	dyn 1:1	28.18 ± 0.37	18.93	22.15	79/21 (29 stolen)	+34%

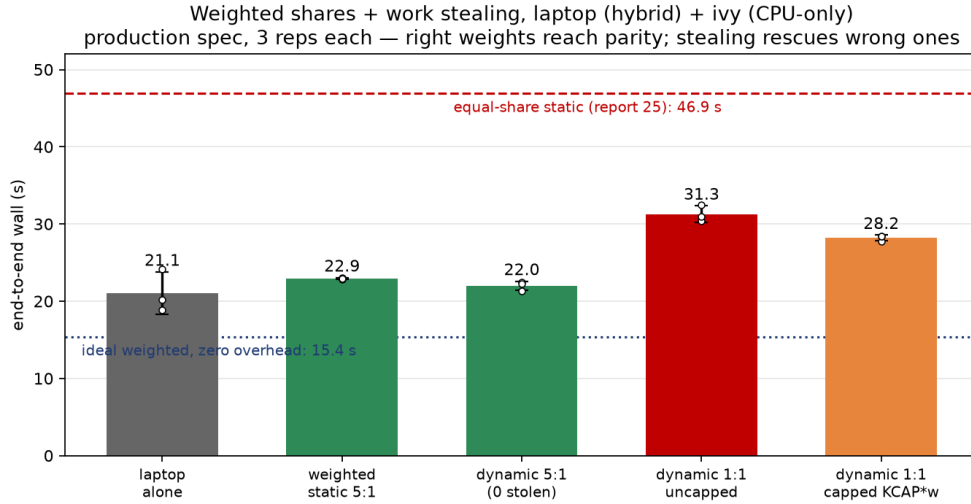


Figure 1: The five walls that tell the story (bars: 3-rep means, whiskers σ , dots reps). Green — calibrated weights, static and dynamic — sits at the grey baseline: parity, the most this pair allows (report 25’s ceiling, dotted line, minus overhead). Red is stealing with wrong 1:1 weights before the chunk cap; orange after it. Everything is far below the dashed line — report 25’s equal-share static at 46.9 s.

Reference points from report 25 (same pair, earlier same-day batch): equal-share cyclic 46.90 ± 1.07 s; ivy solo 85.48 s; ideal weighted wall 15.4 s.

5. Analysis

5.1. Calibrated weights reach baseline parity — the ceiling realized, no more

Weighted static lands at 22.93 ± 0.11 s and dynamic 5:1 at 22.01 ± 0.56 s against a 21.06 ± 2.73 s in-batch baseline — 0.7σ and 0.3σ of the baseline’s own spread: statistically indistinguishable from not distributing at all. This is exactly what report 25’s arithmetic said the pair allows: the ideal weighted wall is 15.4 s, the measured orchestration overhead is 3.1–3.4 s here (e2e minus the slower node’s compute wall: 23.0–19.9 static, 22.0–19.0 dynamic), and the residual is the weighted-random draw’s variance ($\mathcal{S}5.6$). The meaningful comparison is against equal shares: $46.90 \rightarrow 22\text{--}23$ s, a $2.1\times$ improvement from weighting alone. The conclusion of report 25 stands with its constructive half now measured: a $4.56\times$ -slower node cannot shorten this wall, but weighted shares stop it from *lengthening* it — ivy now contributes its 15–17 frames for free instead of costing +150%.

5.2. The unstealable in-flight chunk, and the weight-proportional cap

The uncapped `dyn_11` run is the diagnostic. Its chunk trace shows the defect in one line each: ivy’s first guided chunk was half its 50-frame bag — **25 frames = 27.4 s**

in flight — while the laptop finished its own 50 *plus everything it could steal* (25 frames, in chunks of 13/6/6) in 16.4 s of busy time and then idled ~ 11 s. Stealing moves only *undispatched* frames; a chunk already handed to a node is invisible to the bags, so the slow node’s oversized first chunk bound the wall at 31.27 ± 1.08 s regardless of how well the laptop stole.

The fix (commit `8c21bf4`) prices exposure in time rather than frames: an in-flight chunk of k frames occupies its node for k/rate_r , and rate is proportional to weight, so capping every chunk at $\text{KCAP} \cdot w_r$ frames ($\text{KCAP} = 8$) bounds every node’s unstealable exposure at the *same* wall-time budget — ~ 7 s on both ivy (8×0.855) and the laptop (40×0.188). Measured effect on **dyn_11**: ivy’s busy time $28.0 \rightarrow 22.2$ s, the laptop’s stolen haul $25 \rightarrow 29$ frames, e2e $31.27 \rightarrow 28.18 \pm 0.37$ s (-9.9% , 2.7σ). The fast node keeps its large chunks (few startups on the critical path); only the node whose in-flight work can bind gets fine-grained.

5.3. Stealing recovers 72% of a miscalibration autonomously

With no correct speed information at all (1:1 weights), the capped stealer runs 28.18 s where equal-share static ran 46.90 s — 72% of the 25.8 s penalty recovered with zero configuration (67% against the more favourable uncapped-batch baseline; the claim survives the drift). The mechanism: the laptop empties its 50-frame bag in ~ 9 s, then strips ivy’s bag from the tail in weight-capped chunks, ending at 79/21 — close to the 82/18 optimum the weights failed to encode. The residual +34% over baseline decomposes into ivy’s still-inflated share (21 frames $\times$ deep-draw cost = 22.2 s busy vs. 16.3 s at 5:1) and 6.0 s of per-chunk dispatch overhead on the binding node (e2e — ivy busy; ~ 3 chunk boundaries $\times$ SSH+scp+startup) — the price of the fine granularity that made the rescue possible. Stealing is a recovery mechanism, not a substitute for calibration: it converges toward the right split at chunk granularity, paying dispatch overhead for every correction step.

5.4. With calibrated weights, stealing never fires — and costs nothing

In all three **dyn_51** reps the steal count is zero: the 83/17 deal finishes within ~ 2.7 s on both nodes (laptop busy 18.96, ivy 16.31 s), so no bag empties while another still holds frames. Dynamic mode at correct weights therefore measures pure chunking overhead vs. one-shot weighted static: 22.01 vs. 22.93 s — a wash inside noise (the dynamic mean is actually *lower*; its guided first chunks also start the laptop’s largest work earlier). The implication: there is no premium for running the stealer as the default — it is free insurance that activates only when the weights are wrong (**S5.3**) or the hardware changes out from under them (**S5.5**).

5.5. The live corrective steal: weights are configuration-dependent

The production identity check ran both nodes CPU-pinned — and there the laptop is only $\sim 2.4 \times$ ivy (0.35 vs. 0.855 s/frame), not $4.56 \times$: the 5:1 weights that are correct for the hybrid *overload* a CPU-pinned laptop. The stealer corrected it unprompted: ivy finished its 17 frames and stole 5 back from the laptop’s tail (**[dyn] rank 1 chunk 3 (steal0): 5 frames in 6.18s**), and the run stayed 100/100 byte-identical. This is

the cleanest demonstration of what the dynamic layer is *for*: the right weights are not a property of the machines but of the (machine, executor-mix) pair — the same laptop is worth 5 ivys with its GPU and 2.4 without — and stealing absorbs exactly that model error, in either direction.

5.6. Two noise sources: the fixed random draw, and baseline drift

Two honest caveats. First, the weighted-random draw has real variance at small shares: ivy’s seed-1234 15-frame share costs $19.9\text{ s} = 1.32\text{ s/frame}$ — 55% above its 0.855 population mean, because 15 draws from a $32\times$ -spread cost distribution can land deep-heavy, and the fixed seed makes this *bias, not noise*, across reps ($\sigma = 0.11\text{ s}$ on a mis-centred mean). A deterministic weighted interleave would cut the variance; with stealing on top it is moot (the dynamic tail absorbs it). Second, the laptop baseline drifted across this day’s batches ($18.75 \pm 0.34 \rightarrow 19.20 \pm 0.55 \rightarrow 21.06 \pm 2.73\text{ s}$, one 24.1 s rep — sustained benchmarking thermals), which is why every Δ in Table 1 is computed against its own batch’s baseline and why parity, not a signed few-percent delta, is the defensible claim.

6. Recommendations

1. **Default heterogeneous runs to `dist_dynamic.sh` with measured weights.** Calibrate from solo walls (here $85.48/18.75 \approx 4.6 \Rightarrow \text{WEIGHTS}=5,1$); expect parity with the fast node alone (22.0 vs. 21.1 s) and $2.1\times$ over equal shares. The stealing layer costs nothing when the weights are right (0 steals, -0.9 s vs. static, within noise).
2. **Keep the weight-proportional chunk cap (KCAP = 8).** It is worth -9.9% (2.7σ) on the miscalibrated case by bounding unstealable exposure at $\sim 7\text{ s/node}$. Trade-off: more chunk boundaries on the binding node (6.0 s dispatch overhead in `dyn_11`); lowering KCAP below 8 buys finer rescue granularity only if per-chunk overhead ($\sim 1.5\text{--}2\text{ s}$: SSH, scp, process start) comes down first.
3. **Do not encode weights per machine; encode them per (machine, executor-mix).** The same pair is 4.56:1 hybrid and 2.4:1 CPU-pinned (S5.5); a hosts file that changes `gpuEnable` must change weights too — or lean on the stealer, which corrected the 5:1/CPU-pinned mismatch by exactly 5 frames on its own.
4. **For static-only runs at small shares, consider a deterministic weighted interleave.** The seeded random draw put +55% on ivy’s 15-frame share as a fixed bias (S5.6). Not worth building while the stealer exists; worth remembering if `DIST_WEIGHTS` is ever used bare on a slow node.
5. **Report-25 conclusions stand.** Parity is the ceiling for this pair; a slow node buys capacity, not latency. The now-open measurement is the same weighted/dynamic A/B on comparable nodes, where the ideal is $\sim 2\times$ rather than $1.22\times$ and the interleave study (report 13’s earmark) becomes readable.

7. Conclusion

The two schedulers report 25's arithmetic called for are built, validated, and measured. The seeded weighted-random assignment gives every node a speed-proportional random share with zero communication and keeps the distribution 100/100 byte-identical at production scale; the coordinator stealer adds per-chunk dispatch through the new `DIST_FRAMES` interface and moves undispached tails to whichever node runs dry. With calibrated 5:1 weights, both modes sit at baseline parity (22.9/22.0 vs. 21.1 ± 2.7 s) — the report-25 ceiling realized, and $2.1\times$ better than equal shares. With wrong weights, stealing recovers 72% of the penalty on its own, but only once chunks are capped at $\text{KCAP} \cdot w_r$ frames — the uncapped run measured why: a slow node's 25-frame half-bag chunk sat unstealable for 27.4 s and bound the wall. And the CPU-pinned identity run showed the stealer doing its real job unasked: correcting weights that were right for one executor mix and wrong for another, five frames at a time, byte-identically.